LAPORAN TUGAS TENGAH SEMESTER (UTS)
ARSITEKTUR PERANGKAT LUNAK



Nama : Muh. Ridho Awal Putra (13020240003)
Muh Iswan (130202040013)
Nendra Rizkullah Izzatul Ibad (130320240036)

Kelas : C3

Dosen : Mardiyyah Hasnawi, S.Kom., M.T.,MTA

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS ILMU KOMPUTER

UNIVERSITAS MUSLIM INDONESIA

MAKASSAR

2026

Daftar Isi

BAB I: PENDAHULUAN (U1) 3

BAB II: SKENARIO ATRIBUT KUALITAS (U2) 6

BAB III: TAKTIK DAN POLA ARSITEKTUR (U3–U4) 8

BAB IV: DESAIN DAN DIAGRAM (U5) 11

BAB V: EVALUASI DAN ADR (U6–U7) 14

BAB VI: PENUTUP 16

BAB I: PENDAHULUAN (U1)

1.1 Latar Belakang dan Konteks Sistem

Subsistem **Attendance Management** (Layanan Presensi dan Rekap Kehadiran Smart Campus UMI) adalah modul inti yang bertanggung jawab untuk mencatat, mengelola, memvalidasi, serta menyajikan rekapitulasi kehadiran perkuliahan bagi mahasiswa dan dosen secara terpadu di lingkungan Universitas Muslim Indonesia (UMI). Layanan ini dikembangkan untuk menjawab berbagai kendala operasional akademik yang ditemukan di lapangan berdasarkan hasil analisis wawancara langsung dengan para dosen dan mahasiswa.

1.2 Masalah Utama dan Bukti Wawancara

Berdasarkan hasil investigasi dan wawancara terhadap responden (R1–R5), diperoleh masalah utama yang disepakati untuk diselesaikan dalam perancangan arsitektur ini:

- **Masalah Utama:** Ketidaksesuaian, lambatnya sinkronisasi, dan fragmentasi informasi rekap kehadiran mahasiswa antar-mata kuliah (yang selama ini harus dibuka satu per satu), serta ketidaklengkapan status presensi akibat beberapa perkuliahan belum atau terlambat ter-input ke dalam sistem SIAKAD/Kalam UMI.
- **Ringkasan Bukti Wawancara Pendukung:**
    1. **R1 (Firsa Amelia Putri):** Menyatakan bahwa gangguan jaringan atau sistem sering menghambat pengisian presensi, sehingga dosen membutuhkan rekap otomatis, fitur pencarian/filter, notifikasi kehadiran rendah, serta unduhan laporan (PDF/Excel).
    2. **R2 & R5 (Mahasiswa - Wahyuningsih & Putri Zalsabila):** Mengeluhkan lamanya waktu pengecekan karena informasi rekap harus dibuka per mata kuliah satu per satu dan belum tersedia ringkasan keseluruhan (persentase, izin, sakit, alpa) dalam satu halaman dasbor yang terpusat.

3. **R3 (Mahasiswa - Harry):** Menilai dasbor saat ini terlalu rumit, tidak simpel, dan sering mendapati mata kuliah yang kosong dari data kehadiran.
4. **R4 (Mahasiswa - Salwa):** Mengalami kesulitan pemantauan karena tidak semua dosen mengabsen langsung di SIAKAD UMI, sehingga memerlukan penyempurnaan fitur pencarian dan tampilan rekap.

1.3 Empat Stakeholder Utama

1. **Mahasiswa:** Pengguna utama yang memerlukan pemantauan ringkasan rekap kehadiran dan persentase kehadiran seluruh mata kuliah secara transparan dalam satu halaman.
2. **Dosen:** Pengguna yang mengelola presensi kelas secara langsung, memantau keaktifan mahasiswa, serta mengunduh laporan rekap berkala.
3. **Program Studi / BTIK UMI:** Pihak pengelola institusional yang memantau tingkat kepatuhan dan efektivitas proses perkuliahan secara menyeluruh.
4. **Bagian Administrasi Akademik:** Pengelola data master (mahasiswa, dosen, KRS, dan penjadwalan akademik universitas).

1.4 Tiga Architectural Significant Requirements (ASR)

1. **ASR-01 (Performa Rekapitulasi):** Halaman rekapitulasi seluruh mata kuliah harus dapat dimuat dalam waktu ≤ 2 detik meskipun diakses secara serentak oleh ± 1.000 pengguna pada jam sibuk perkuliahan.
2. **ASR-02 (Ketersediaan & Toleransi Jaringan):** Sistem harus tetap dapat diakses dan responsif di tengah kondisi jaringan internet seluler/kampus yang fluktuatif melalui strategi penyimpanan lokal (*offline-first*) dan mekanisme *caching*.
3. **ASR-03 (Modifiabilitas & Integrasi Aman):** Penambahan fitur baru atau pemeliharaan modul tidak boleh mengganggu kestabilan basis data transaksional utama SIAKAD UMI.

1.5 Tiga Karakteristik Pendorong (Quality Attribute Drivers)

1. **Performance / Latency:** Menjamin kecepatan respons akses data rekap saat beban puncak.
2. **Usability / Accessibility:** Menyediakan antarmuka yang sederhana, intuitif, dan terpusat (*single-page dashboard*).
3. **Availability & Consistency:** Menjaga ketersediaan layanan dan konsistensi data di tengah kendala konektivitas yang tidak stabil.

BAB II: SKENARIO ATRIBUT KUALITAS (U2)

Bab ini menguraikan skenario atribut kualitas sistem *Attendance Management* UMI yang diturunkan dari karakteristik pendorong utama sistem. Setiap skenario dijabarkan ke dalam enam komponen standar arsitektur perangkat lunak beserta ukuran respons yang terukur secara kuantitatif.

2.1 Skenario 1 (Karakteristik: *Performance* — Menjawab Masalah Utama)

Skenario ini dirancang untuk mengatasi masalah utama terkait lambatnya sinkronisasi dan fragmentasi informasi rekap kehadiran yang sebelumnya harus dibuka satu per satu.

- **Source (Sumber):** Mahasiswa aktif tingkat akhir yang ingin memantau status kehadiran.
- **Stimulus (Rangsangan):** Mengirimkan permintaan HTTP GET /rekap melalui portal mahasiswa untuk melihat rekapitulasi kehadiran seluruh mata kuliah secara terpusat.
- **Artifact (Artefak):** Modul *Rekap Service*, *API Gateway*, dan lapisan *Cache*.
- **Environment (Lingkungan):** Kondisi jam sibuk perkuliahan dengan beban akses bersamaan sebanyak ± 1.000 pengguna aktif.
- **Response (Respons):** Sistem mengambil data agregat dari *cache/read model* dan merender dasbor rekapitulasi secara utuh tanpa hambatan.
- **Response Measure (Ukuran Respons):** Waktu muat halaman rekapitulasi keseluruhan mata kuliah tampil dalam waktu ≤ 2 **detik**.

2.2 Skenario 2 (Karakteristik: *Usability*)

Skenario ini berfokus pada kemudahan navigasi bagi pengguna agar tidak lagi kesulitan mencari informasi kehadiran.

- **Source (Sumber):** Mahasiswa baru pengguna sistem akademik.

- **Stimulus (Rangsangan):** Melakukan navigasi ke dalam portal akademik untuk mencari informasi jumlah kehadiran, izin, sakit, alpa, dan persentase kehadiran suatu mata kuliah.
- **Artifact (Artefak):** Antarmuka Pengguna (*UI Gateway* / Dasbor Presensi Tunggal).
- **Environment (Lingkungan):** Kondisi operasional normal pada perangkat telepon seluler (*smartphone*).
- **Response (Respons):** Sistem menampilkan menu pencarian dan menyajikan tabel rekap yang ringkas, intuitif, serta terpusat dalam satu halaman.
- **Response Measure (Ukuran Respons):** Pengguna berhasil menemukan total kehadiran dan persentase seluruh mata kuliah dalam waktu ≤ 2 **klik** tanpa bantuan manual.

2.3 Skenario 3 (Karakteristik: *Availability & Consistency*)

Skenario ini dirancang untuk mengatasi kendala teknis jaringan internet yang tidak stabil di lingkungan kampus.

- **Source (Sumber):** Dosen pengampu mata kuliah.
- **Stimulus (Rangsangan):** Melakukan penyimpanan data presensi kelas saat kondisi jaringan internet seluler/kampus mengalami gangguan atau putus-putus.
- **Artifact (Artefak):** Modul penyimpanan lokal (*Local Storage / Progressive Web App*) dan *Sync Engine*.
- **Environment (Lingkungan):** Jaringan internet kampus tidak stabil.
- **Response (Respons):** Sistem mencatat data presensi secara lokal di perangkat, menampilkan indikator status *offline*, dan melakukan sinkronisasi otomatis begitu koneksi internet pulih.
- **Response Measure (Ukuran Respons):** Data presensi aman tanpa kehilangan catatan, dan proses sinkronisasi ke server pusat selesai dalam waktu ≤ 1 **menit** setelah jaringan tersambung kembali.

BAB III: TAKTIK DAN POLA ARSITEKTUR (U3–U4)

Bab ini membahas taktik arsitektur yang digunakan untuk merespons skenario atribut kualitas serta pemilihan pola arsitektur guna mendukung perancangan subsistem *Attendance Management* UMI.

3.1 Taktik Arsitektur (U3)

Taktik-taktik berikut diterapkan untuk memenuhi target atribut kualitas sekaligus mengatasi masalah utama fragmentasi dan lambatnya sinkronisasi data presensi:

3.1.1 Taktik untuk Skenario 1 (Performance / Waktu Respons Rekap)

- **Taktik yang Digunakan:** *Caching* dan *Read Model* Terisolasi.
- **Cara Kerjanya:** Alih-alih melakukan *query* agregasi secara langsung ke basis data transaksional utama setiap kali mahasiswa meminta rekapitulasi, sistem memanfaatkan lapisan memori *cache* (*Cache-Aside pattern*). Data rekapitulasi per mahasiswa dihitung sebelumnya dan disimpan dalam model baca (*read model*) khusus yang dioptimalkan untuk operasi GET.
- **Harga yang Dibayar (Trade-off):**
    1. Memunculkan potensi jeda kesegaran data sesaat (*eventual consistency*) jika ada pembaruan presensi yang belum disinkronkan ke dalam *cache*.
    2. Menambah kompleksitas pengelolaan memori dan strategi invalidasi *cache* pada sisi arsitektur backend.

3.1.2 Taktik untuk Skenario 2 (Usability / Kemudahan Akses Dasbor)

- **Taktik yang Digunakan:** *UI Component Aggregation* / *Single-Page Dashboard Design*.
- **Cara Kerjanya:** Menggabungkan berbagai data kehadiran dari berbagai mata kuliah ke dalam satu antarmuka dasbor terpusat (*Single-Page Application*). Sistem menyediakan kolom pencarian (*search bar*) dan filter instan sehingga mahasiswa dapat langsung melihat ringkasan seluruh mata kuliah, persentase kehadiran, izin, sakit, dan alpa tanpa harus membuka menu per mata kuliah satu per satu.
- **Harga yang Dibayar (Trade-off):**
    1. *Payload* data yang dikirimkan dari server ke klien menjadi lebih besar dalam satu kali muat (*initial load*).
    2. Memerlukan desain tata letak antarmuka yang responsif agar tetap ringan ketika diakses melalui perangkat telepon seluler (*smartphone*).

3.1.3 Taktik untuk Skenario 3 (Availability & Consistency / Toleransi Jaringan)

- **Taktik yang Digunakan:** *Local Queue* / *Offline-First Architecture* (*Progressive Web App*).
- **Cara Kerjanya:** Ketika jaringan internet mengalami gangguan atau terputus saat proses pencatatan presensi, sistem secara otomatis mengamankan data transaksi ke dalam penyimpanan lokal perangkat (*local storage*). Sebuah *sync engine* di latar belakang memantau status koneksi dan mengirimkan ulang data yang tertunda secara otomatis begitu koneksi internet kembali stabil.
- **Harga yang Dibayar (Trade-off):**
    1. Meningkatkan kompleksitas pada sisi klien untuk menangani potensi konflik data ganda (*duplicate submission*) saat sinkronisasi ulang dilakukan.
    2. Data *offline* yang tersimpan di perangkat berisiko usang jika tidak disertai penanda waktu (*timestamp*) yang valid.

3.2 Pola Arsitektur (U4)

Dua pola arsitektur dari kategori berbeda dipilih untuk mendukung skalabilitas dan komunikasi asinkron dalam sistem:

3.2.1 Pola P-01 — Microservices Architecture (Layered & Component-Based)

- **Kategori:** Struktur/pembagian lapisan dan distribusi layanan (*Structural / Distribution*).
- **Definisi:** Pola arsitektur yang menyusun aplikasi sebagai kumpulan layanan kecil yang independen, berjalan dalam prosesnya sendiri dan berkomunikasi melalui mekanisme ringan (API HTTP/REST).
- **Alasan Sesuai Kasus UMI:** Mengatasi masalah utama fragmentasi rekap presensi dan kelambatan sinkronisasi data dari berbagai mata kuliah di SIAKAD, sehingga modul presensi dapat dikembangkan, diskalakan, dan dipelihara secara terisolasi tanpa mengganggu modul akademik lainnya.
- **Kelebihan:** Memungkinkan skalabilitas mandiri pada layanan rekapitulasi presensi yang sering diakses mahasiswa secara bersamaan, serta melokalisasi dampak kesalahan (*fault isolation*).
- **Kekurangan:** Meningkatkan kompleksitas operasional, pengelolaan jaringan antar-layanan (*network latency*), serta *overhead* koordinasi transaksi data lintas layanan.

3.2.2 Pola P-02 — Publish–Subscribe (Event-Driven Architecture)

- **Kategori:** Komunikasi/integrasi berbasis *event* (*Messaging / Communication*).
- **Definisi:** Pola komunikasi asinkron di mana pengirim (*publisher*) memancarkan *event* ke sebuah *broker* tanpa harus mengetahui siapa penerima (*subscriber*) yang mendengarkannya.
- **Alasan Sesuai Kasus UMI:** Dibutuhkan untuk mengirimkan notifikasi instan atau peringatan (*alert*) ketika tingkat kehadiran mahasiswa terdeteksi berada di bawah batas minimum, tanpa membuat proses utama pemuatan rekap menjadi lambat.
- **Kelebihan:** Mengurangi keteraitan langsung (*loose coupling*) antar-komponen, sehingga penambahan fitur notifikasi atau pelaporan baru tidak mengganggu kinerja inti layanan presensi.
- **Kekurangan:** Potensi keterlambatan sinkronisasi sesaat (*eventual consistency delay*) pada pengiriman notifikasi akibat antrean pesan, serta kesulitan dalam melacak urutan *debugging* pesan asinkron.

3.2.3 Perbedaan Gaya dan Pola Arsitektur

- **Gaya (Style):** Karakteristik umum atau cetak biru tingkat makro mengenai cara keseluruhan sistem diorganisasikan (misalnya gaya terdistribusi atau *layered*).
- **Pola (Pattern):** Solusi berulang yang lebih spesifik pada tingkat desain komponen atau taktik komunikasi untuk mengatasi masalah rekayasa perangkat lunak tertentu.
- **Penerapan pada Sistem UMI:** Gaya arsitektur sistem secara umum menggunakan gaya terdistribusi berbasis *Microservices*, sedangkan pola arsitektur yang diterapkan di dalamnya adalah pola *API Composition* dan *Publish-Subscribe* untuk menyelesaikan spesifik masalah fragmentasi rekap presensi.

BAB IV: DESAIN DAN DIAGRAM (U5)

Bab ini menyajikan rincian perancangan desain arsitektur dan diagram visual yang merepresentasikan struktur komponen, aliran komunikasi, serta perilaku proses pada subsistem *Attendance Management* UMI.

4.1 Diagram Konteks

Diagram konteks menggambarkan batasan sistem (*system boundary*) serta interaksi antara subsistem *Attendance Management* dengan para aktor eksternal yang terlibat secara langsung di dalam lingkungan kampus UMI.

- **Aktor Utama:** Mahasiswa, Dosen, Program Studi / BTIK UMI, dan Bagian Administrasi Akademik.
- **Alur Interaksi:**
    1. Mahasiswa berinteraksi untuk memantau rekapitulasi kehadiran dan persentase kehadiran seluruh mata kuliah.
    2. Dosen berinteraksi untuk melakukan penginputan, validasi, dan pengunduhan laporan rekap kelas.
    3. Bagian Administrasi Akademik dan BTIK menyuplai data master akademik (KRS, jadwal, dan data mahasiswa) serta memantau kepatuhan pelaporan.

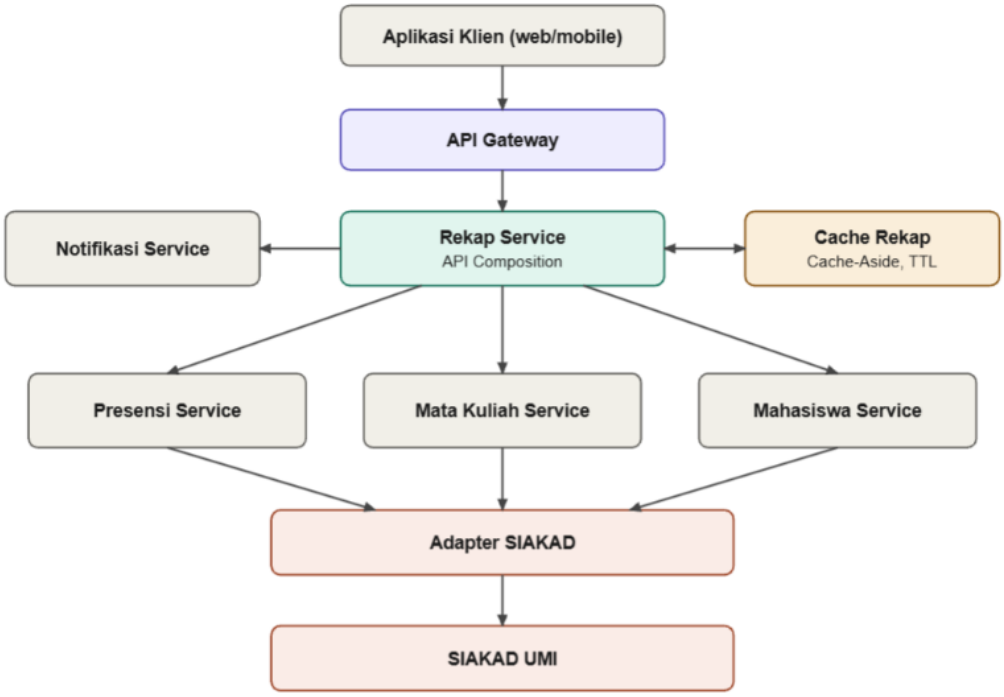


4.2 Diagram Arsitektur

Diagram arsitektur memetakan struktur komponen perangkat lunak secara terdistribusi berdasarkan pola *Microservices* dan *Publish-Subscribe* yang telah dibahas pada bab sebelumnya.

- **Komponen Utama dalam Diagram:**
    1. **Client Layer:** Portal web dan aplikasi seluler mahasiswa/dosen (*Progressive Web App*).
    2. **API Gateway / Load Balancer:** Gerbang utama yang mengatur lalu lintas permintaan HTTP dan keamanan akses.

3. **Attendance & Recap Service:** Layanan inti yang memproses logika bisnis pencatatan presensi dan agregasi data rekap.
4. **Cache & Read Model Layer:** Lapisan memori terisolasi untuk mempercepat akses penarikan rekapitulasi secara instan (≤ 2 detik).
5. **Message Broker / Event Bus:** Saluran komunikasi asinkron untuk penyiaran *event* peringatan kehadiran rendah (*Publish-Subscribe*).
6. **SIAKAD Integration Adapter:** Modul penghubung aman yang menyelaraskan data dengan basis data transaksional utama SIAKAD UMI.



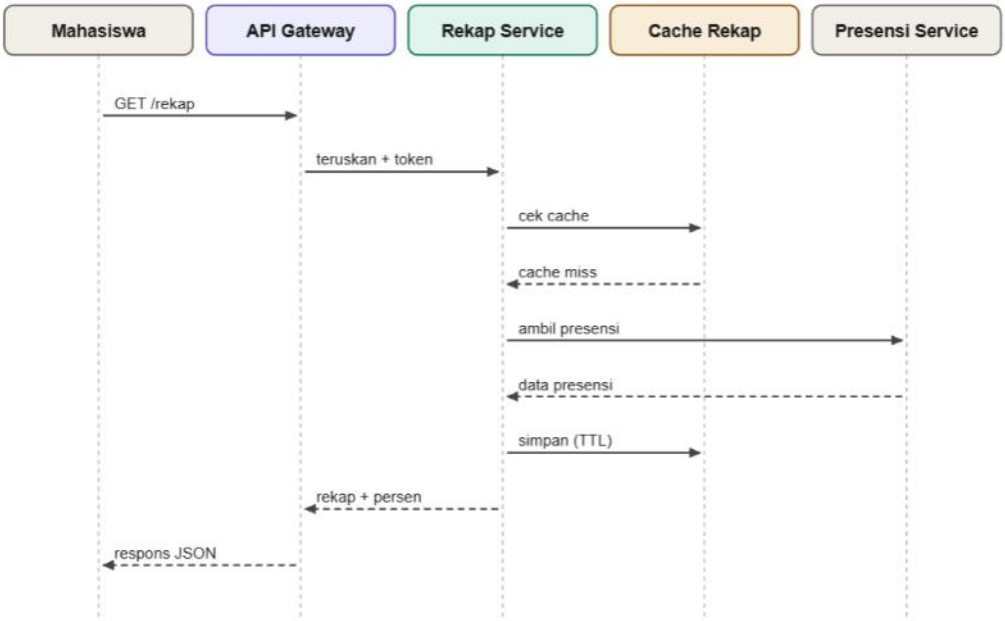
4.3 Diagram UML Activity

Diagram aktivitas (*UML Activity Diagram*) menggambarkan alur kerja sistem secara rinci, khususnya skenario proses pencatatan kehadiran oleh dosen di tengah kondisi jaringan yang fluktuatif serta proses pengecekan rekapitulasi oleh mahasiswa.

- **Alur Utama Aktivitas:**
  1. Pengguna (Mahasiswa/Dosen) mengakses portal *Attendance Management*.
  2. Sistem memeriksa status koneksi jaringan:
     - Jika **Online**: Sistem langsung mengambil/menyimpan data secara *real-time* ke server pusat melalui *API Gateway*.

- Jika **Offline**: Sistem mengamankan transaksi data presensi ke penyimpanan lokal perangkat (*local storage*) dan mengaktifkan indikator *offline*.

3. Begitu jaringan kembali terhubung, *Sync Engine* secara otomatis melakukan sinkronisasi data tertunda ke basis data pusat tanpa kehilangan catatan.



BAB V: EVALUASI DAN ADR (U6–U7)

Bab ini menyajikan hasil evaluasi arsitektur sistem menggunakan metode pohon utilitas, analisis perbandingan alternatif, metode ATAM (*Architecture Tradeoff Analysis Method*), serta pencatatan keputusan arsitektur (*Architecture Decision Records*).

5.1 Evaluasi Arsitektur (U6)

5.1.1 Pohon Utilitas Mini — Tiga Skenario

Pohon utilitas digunakan untuk menentukan atribut kualitas prioritas dalam pengembangan *Attendance Management* UMI:

- **Performa (SK-01) — [High, High]:** Saat jam sibuk dengan ± 1.000 mahasiswa membuka rekap secara bersamaan, seluruh mata kuliah harus tampil dalam waktu maksimal 2 detik.
- **Usability (SK-02) — [High, Medium]:** Mahasiswa baru dapat menemukan jumlah dan persentase kehadiran seluruh mata kuliah dalam waktu ≤ 2 klik tanpa bantuan.
- **Availability & Konsistensi (SK-03) — [High, High]:** Setelah presensi disimpan, rekap diperbarui maksimal 1 menit. Jika jaringan terputus, presensi diamankan dalam antrean lokal dan disinkronkan otomatis saat kembali *online*.

5.1.2 Perbandingan Dua Alternatif Rancangan

- **Alternatif A (Modul Rekap di Dalam SIAKAD):** Mengintegrasikan modul rekap langsung dengan *query* ke basis data transaksional utama.
- **Alternatif B (Layanan Rekap Terpisah dengan Read Model):** Memisahkan layanan rekap kehadiran menggunakan *read model* dan *cache*.
- **Alternatif Terpilih: Alternatif B** dipilih karena unggul dalam hal performa, ketersediaan (*availability*), dan fleksibilitas pengembangan fitur tanpa membebani basis data utama SIAKAD.

5.1.3 Analisis ala ATAM

- **Risiko 1:** *Read model* menggunakan *eventual consistency*, sehingga data rekap berpotensi mengalami keterlambatan sesaat. *(Mitigasi: Sinkronisasi berbasis event dan rekonsiliasi harian)*.
- **Risiko 2:** Rekap tidak lengkap jika dosen belum mengisi presensi di SIAKAD. *(Mitigasi: Kebijakan kampus dan penanda khusus mata kuliah)*.

- **Non-risiko 1:** Layanan rekap bersifat *read-only*, sehingga gangguan pada layanan rekap tidak menghambat proses pencatatan presensi utama di SIAKAD.
- **Sensitivity Point 1:** Interval sinkronisasi dan TTL *cache* sangat memengaruhi kesegaran informasi dan beban sistem (diatur maksimal 1 menit).
- **Trade-off Point 1:** Pemisahan layanan meningkatkan performa dan ketersediaan, namun mengurangi konsistensi data secara instan serta menambah kompleksitas operasional.

5.1.4 Kesimpulan Evaluasi Masalah Utama

Rancangan Alternatif B terbukti mampu mengatasi sebagian besar masalah utama seperti fragmentasi rekap per mata kuliah, kesulitan navigasi, kendala performa saat jam sibuk, serta mitigasi jaringan tidak stabil melalui antrean lokal

.

5.2 Architecture Decision Records (U7)

Berikut adalah ringkasan dokumen keputusan arsitektur (*Architecture Decision Records*) yang mendasari perancangan sistem:

ADR-001: Layanan Rekap Kehadiran Terpisah dengan Read Model

- **Status:** Accepted
- **Context:** Kebutuhan akses rekap terpusat bagi mahasiswa tanpa membebani kinerja basis data transaksional SIAKAD utama.
- **Decision:** Membangun layanan rekap kehadiran tersendiri yang menyimpan data agregat dalam *read model* terpisah khusus operasi baca cepat.
- **Consequences:** Akses rekap menjadi sangat cepat dan skalabel, namun memunculkan potensi *eventual consistency* serta penambahan komponen yang harus dikelola.

ADR-002: Sinkronisasi Berbasis Event dengan Rekonsiliasi Harian

- **Status:** Accepted
- **Context:** Kebutuhan pembaruan data yang cepat namun tetap menjamin keakuratan data tanpa kehilangan pesan *event*.
- **Decision:** Setiap presensi memancarkan *event* pembaruan *read model* (≤ 1 menit), didukung *job* rekonsiliasi otomatis setiap dini hari untuk mengoreksi selisih data.
- **Consequences:** Data tersaji secara *near real-time* dengan konsistensi terjaga, namun memerlukan infrastruktur *message queue* dan penanganan duplikasi data.

BAB VI: PENUTUP

6.1 Kesimpulan

Berdasarkan perancangan dan evaluasi arsitektur perangkat lunak pada subsistem *Attendance Management* (Layanan Rekap Kehadiran Smart Campus UMI), dapat disimpulkan bahwa:

- Rancangan arsitektur berbasis *Microservices* dan penggunaan *Read Model* terpisah (*Alternatif B*) terbukti efektif menyelesaikan masalah utama fragmentasi informasi rekap kehadiran mahasiswa dan kelambatan sinkronisasi data dari SIAKAD.
- Penerapan taktik *caching* dan antrean lokal (*offline-first*) mampu memastikan sistem tetap responsif di tengah kendala performa jam sibuk (± 1.000 pengguna) serta kondisi jaringan internet kampus yang fluktuatif.
- Keputusan perancangan telah divalidasi melalui pohon utilitas, perbandingan alternatif berbobot, dan analisis ATAM, serta didokumentasikan secara formal melalui *Architecture Decision Records* (ADR-001 dan ADR-002).

6.2 Saran

Untuk pengembangan dan implementasi sistem ke depan, disarankan hal-hal berikut:

1. Melakukan pengujian nyata (*load testing* dan *stress testing*) untuk memastikan target waktu muat ≤ 2 detik dapat tercapai secara konsisten pada infrastruktur server kampus.
2. Menerapkan kebijakan institusional yang mewajibkan seluruh dosen melakukan penginputan presensi secara tertib langsung melalui SIAKAD UMI guna meminimalkan data kosong.
3. Mengembangkan mekanisme pemantauan otomatis (*monitoring tool*) untuk mengawasi kelancaran sinkronisasi berbasis *event* dan proses rekonsiliasi harian.

DAFTAR PUSTAKA

(Catatan: Anda dapat menyesuaikan atau melengkapi referensi berikut dengan minimal 5 sumber literatur/jurnal/buku terkait arsitektur perangkat lunak)

[1] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 3rd ed. Boston: Addison-Wesley, 2012.

[2] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston: Addison-Wesley, 2002.

[3] R. N. Taylor, N. Medvidovic, and E. M. Dashofy, *Software Architecture: Foundations, Theory, and Practice*. genellikle: John Wiley & Sons, 2009.

[4] P. Clements, F. Bachmann, L. Bass, D. Garlan, J. Ivers, R. Little, P. Nord, and J. Stafford, *Documenting Software Architectures: Views and Beyond*, 2nd ed. Boston: Addison-Wesley, 2010.

[5] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2nd ed. Sebastopol: O'Reilly Media, 2021.